

Curs 3 – Big Data

Procesarea distribuită a datelor

Caracteristicile principale pentru Apache Hadoop

Apache Hadoop characteristics

- Is an open-source, distributed processing framework
- Enables distributed storage and processing for large amounts of data
- Maps tasks to nodes within clusters of servers
- Hadoop components: Hadoop Distributed File System (HDFS), YARN, MapReduce, Hadoop Common
- Clusters consist of main nodes and worker nodes



aws

Hadoop este un **framework open-source** care utilizează o **arhitectură de procesare distribuită** pentru a gestiona volume foarte mari de date. Prin procesare distribuită se înțelege faptul că datele și calculele nu sunt realizate pe un singur sistem, ci sunt împărțite între mai multe servere care lucrează împreună sub forma unui cluster.

În cadrul Hadoop, sarcinile de procesare sunt **împărțite și distribuite automat** către un cluster de servere standard (commodity servers). Fiecare server procesează o parte din date, iar rezultatele sunt reunite ulterior. Această abordare permite procesarea paralelă și creșterea semnificativă a performanței atunci când se lucrează cu seturi mari de date.

Hadoop este alcătuit din **patru componente principale**, fiecare având un rol bine definit:

- **Hadoop Distributed File System (HDFS)** – sistemul de fișiere distribuit, responsabil pentru stocarea datelor;
- **YARN (Yet Another Resource Negotiator)** – componenta care gestionează resursele clusterului și alocă memoria și procesorul aplicațiilor;
- **MapReduce** – modelul de programare utilizat pentru procesarea batch a datelor;
- **Hadoop Common** – bibliotecile și utilitarele de bază necesare funcționării întregului ecosistem Hadoop.

Din punct de vedere arhitectural, un cluster Hadoop este format din **noduri principale (main nodes)** și **noduri de lucru (worker nodes)**. Nodurile principale sunt responsabile de coordonarea și orchestrarea joburilor, de gestionarea metadatelor și a resurselor. Nodurile de lucru execută efectiv sarcinile de procesare și stochează datele.

Beneficiile framework-ului Hadoop

Un avantaj important al Hadoop este capacitatea de a **stoca cantități foarte mari de date**, fără a fi necesară preprocesarea acestora înainte de stocare. Acest lucru oferă flexibilitate ridicată și permite păstrarea datelor brute pentru analize ulterioare. Hadoop oferă un nivel ridicat de **toleranță la erori**. Dacă un nod din cluster cedează, sarcinile acestuia sunt redistribuite automat către celelalte noduri. În plus, pierderea datelor este prevenită prin stocarea mai multor copii ale acelorași date pe noduri diferite din cluster. Infrastructura Hadoop este **scalabilă**, ceea ce înseamnă că, pe măsură ce volumul de date sau cerințele de procesare cresc, pot fi adăugate cu ușurință noi noduri. Fiind un framework open-source, Hadoop este gratuit și poate rula pe hardware relativ ieftin. Un alt beneficiu major este faptul că Hadoop poate procesa **date structurate, semi-structurate și nestructurate**. Datele pot fi transformate în diferite formate și integrate cu seturi de date existente, fiind posibilă stocarea acestora atât cu schemă, cât și fără schemă.

Provocările framework-ului Hadoop - fiind un proiect open-source aflat în continuă evoluție Hadoop prezintă anumite limitări. Hadoop se bazează în mare măsură pe limbajul de programare **Java**, care este frecvent ținta atacurilor de securitate, ceea ce poate expune sistemul la vulnerabilități. De asemenea, **securitatea** reprezintă o provocare importantă. Lipsa mecanismelor implicite de autentificare și criptare poate ridica probleme legate de integritatea și veridicitatea datelor, fiind necesară implementarea

Hadoop Distributed File System (HDFS)

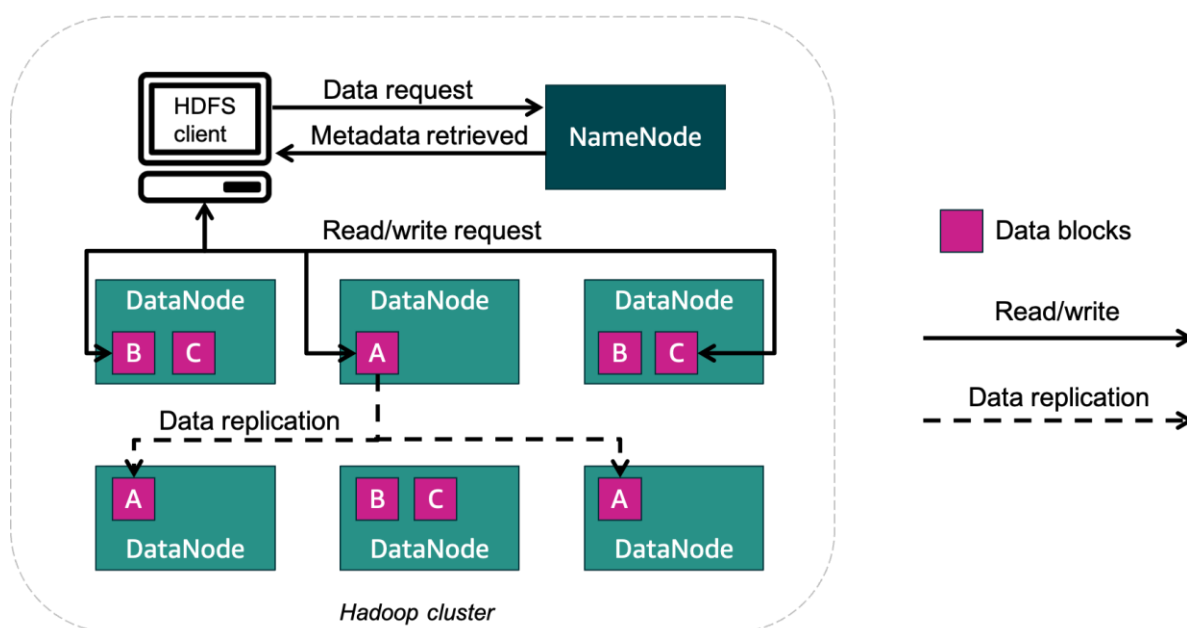


Fig. 1. Hadoop Distributed File System

HDFS (Hadoop Distributed File System) este **sistemul de fișiere distribuit** al framework-ului Hadoop. Rolul său principal este de a **stoca volume foarte mari de date** într-un mod scalabil, fiabil și eficient, astfel încât aceste date să poată fi procesate ulterior de aplicații Big Data, precum **MapReduce sau Apache Spark**. HDFS este construit pentru scenarii în care seturile de date sunt foarte mari, iar un punct forte îl reprezintă **viteză mare de transfer (throughput)** și pe **toleranță la erori**.

Arhitectura HDFS: NameNode și DataNodes

HDFS folosește o **arhitectură ierarhică de tip master-worker**, formată din două tipuri de noduri:

- 1. NameNode (nodul principal)** - este componenta centrală a sistemului HDFS și are rol de **coordonare și administrare**. Acesta păstrează **metadatele** sistemului de fișiere (numele fișierelor, structura directoarelor); știe **în câte blocuri este împărțit fiecare fișier**; știe **pe ce DataNode se află fiecare bloc**; gestionează factorul de replicare și starea nodurilor din cluster.
- 2. DataNodes (nodurile de lucru)** - **stocază efectiv datele**, sub formă de blocuri; execută operații de citire și scriere la cererea aplicațiilor; raportează periodic către NameNode starea lor (heartbeat).

Fiecare dreptunghi „DataNode” conține unul sau mai multe blocuri de date (A, B, C).

Împărțirea fișierelor în blocuri

HDFS este proiectat pentru fișiere foarte mari, atunci când un fișier este salvat acesta este **împărțit automat în blocuri de dimensiune fixă** (de exemplu 128 MB); fiecare bloc este tratat independent. Exemplul: fișierul este împărțit în **trei blocuri: A, B și C**; aceste blocuri sunt distribuite pe mai multe DataNodes din cluster. Această împărțire permite procesarea paralelă a datelor.

Replicarea blocurilor și toleranța la erori

Pentru a preveni pierderea datelor, HDFS utilizează **replicarea blocurilor**: fiecare bloc este copiat pe mai multe DataNodes; numărul de copii se numește **factor de replicare** (implicit este 3); factorul de replicare este configurabil. Se observă că blocul **A** apare pe mai multe DataNodes; la fel și blocurile **B** și **C**. Dacă un DataNode se defectează: NameNode detectează problema; blocurile lipsă sunt recreate automat pe alte noduri, folosind copiile existente; aplicațiile pot continua să funcționeze fără pierderi de date. Acest mecanism oferă **un nivel ridicat de fault tolerance**.

HDFS este un sistem de fișiere distribuit conceput pentru: stocarea fișierelor foarte mari; procesare paralelă eficientă; toleranță ridicată la erori; scalabilitate prin adăugarea de noi noduri. Prin separarea clară între **NameNode (metadata)** și **DataNodes (date efective)**, HDFS oferă o bază solidă pentru arhitecturile Big Data și a stat la baza dezvoltării soluțiilor moderne, precum **Apache Spark și platformele cloud (Databricks)**.

Arhitectura YARN și managementul resurselor într-un cluster Hadoop

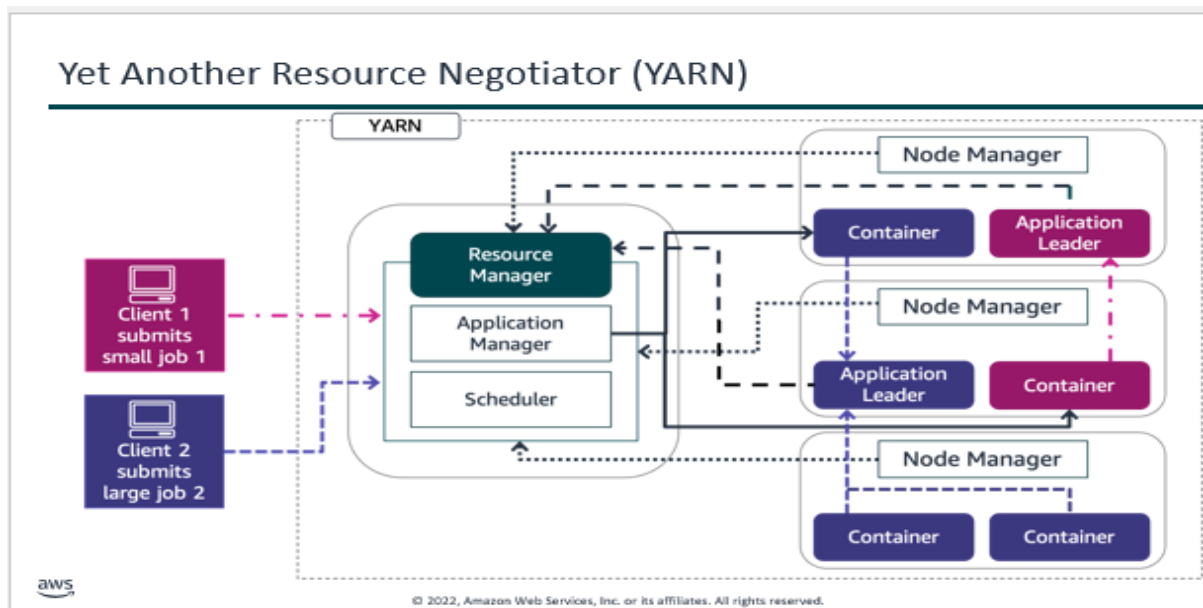


Fig 2. Modul în care **YARN (Yet Another Resource Negotiator)** gestionează execuția mai multor joburi într-un **cluster Hadoop**,

YARN poate fi privit ca un **sistem de operare distribuit**, responsabil cu alocarea și coordonarea resurselor de calcul din cluster. YARN are rolul de a: primi joburi de la utilizatori sau aplicații, decide cum sunt împărțite resursele (CPU, memorie), lansa și monitoriza execuția aplicațiilor, asigura rularea eficientă și echitabilă a mai multor joburi simultan. În această arhitectură, **stocarea datelor** este realizată de HDFS, iar **procesarea** este coordonată de YARN.

Zona YARN - reprezintă **componenta de management al resurselor** din Hadoop. Toate cererile de procesare trec prin această zonă. YARN decide: când pornește un job, câte resurse primește, unde rulează.

Resource Manager este componenta centrală a YARN și are o viziune globală asupra întregului cluster. El primește toate cererile de joburi, știe ce resurse sunt disponibile în cluster, coordonează distribuirea acestora. Resource Manager **nu execută task-uri**, ci doar le planifică. Scheduler-ul este o subcomponentă a Resource Manager-ului. Rolul său este de a: aloca resurse aplicațiilor, decide ordinea de execuție, asigura o distribuție echitabilă a resurselor între joburi mici și mari. Astfel, mai multe aplicații pot rula simultan fără a bloca clusterul.

Application Manager gestionează ciclul de viață al aplicațiilor, acceptă joburile trimise de clienți, lansează primul container al aplicației, se ocupă de relansare în caz de eșec. Această componentă asigură că fiecare aplicație este inițializată corect.

Node Manager rulează pe fiecare nod de lucru din cluster. Rolul său este local și include: gestionarea resurselor nodului (CPU, memorie), lansarea și monitorizarea containerelor, raportarea stării către Resource Manager. Fiecare nod de lucru are propriul Node Manager.

Pentru fiecare aplicație care rulează în cluster există un Application Leader. Acesta comunică cu Resource Manager pentru a cere resurse, coordonează task-urile aplicației, monitorizează progresul execuției. **Application Leader** este specific fiecărei aplicații, nu întregului cluster.

Containerele reprezintă **unități de alocare a resurselor**. Un container conține: o cantitate de memorie, o cotă de procesor. În container rulează efectiv task-urile aplicațiilor. Un job mare va utiliza mai multe containere, distribuite pe mai multe noduri.

Cum funcționează sistemul (flux logic)

- Un client trimite un job către cluster.
- Resource Manager primește cererea.
- Application Manager pornește Application Leader.
- Scheduler-ul alocă resursele necesare.
- Node Manager-ele lansează containerele.
- Task-urile rulează în paralel în containere.
- Aplicația finalizează execuția.

Această arhitectură este utilizată de aplicații Big Data (MapReduce, Spark, Hive), clustere Hadoop clasice, medii on-premises sau cloud (de exemplu, AWS EMR).

YARN este componenta care permite rularea simultană a mai multor aplicații într-un cluster Hadoop, asigurând utilizarea eficientă a resurselor. Deși în platformele moderne acest model este abstractizat sau înlocuit, YARN rămâne esențial pentru înțelegerea principiilor de bază ale procesării distribuite și ale managementului resurselor în Big Data.

Procesarea datelor cu Hadoop MapReduce

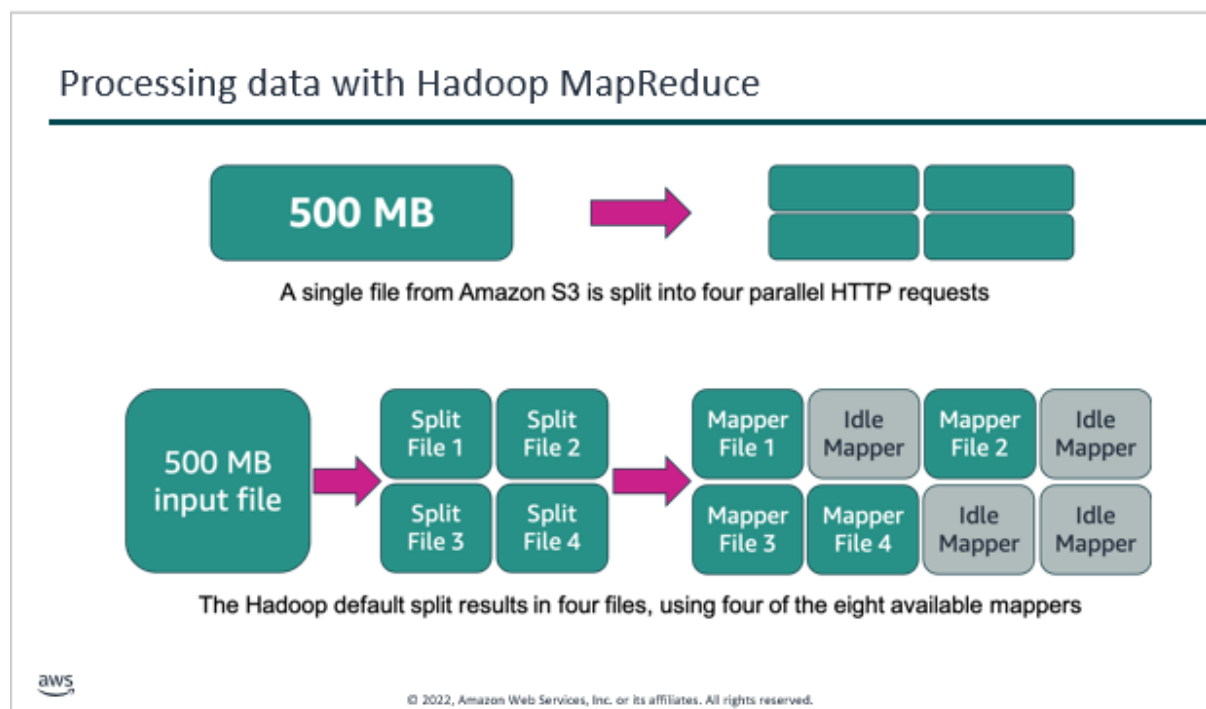


Fig 3. Procesarea datelor cu Hadoop MapReduce

Această schemă arată că **Hadoop MapReduce** procesează datele prin împărțirea fișierelor în segmente, alocarea fiecărui segment unui mapper și executarea în paralel a aceleiași logici de procesare. Acest model face posibilă analiza volumelor mari de date, dar vine și cu anumite limitări legate de flexibilitate și utilizarea optimă a resurselor.

Procesul începe cu un **fișier de intrare de 500 MB**, stocat într-un sistem de stocare distribuit, în acest caz Amazon S3. Hadoop este proiectat pentru a lucra cu fișiere mari, motiv pentru care nu procesează fișierul ca un întreg, ci îl împarte automat în mai multe părți mai mici. Această etapă poartă numele de **splitting**.

Fișierul de 500 MB este împărțit în **patru segmente distincte**, numite *splits*. Fiecare split reprezintă o porțiune din fișierul original și poate fi procesată independent de celelalte. Această împărțire permite accesarea datelor prin mai multe cereri paralele, ceea ce crește viteza de citire și procesare. După împărțirea fișierului, fiecare split este alocat unui **mapper**. Mapper-ul este un proces care execută funcția Map asupra datelor primite. În cluster există opt mapper-e disponibile, însă, deoarece există doar patru splits, **doar patru mapper-e sunt utilizate efectiv**, iar celelalte rămân inactive. Acest aspect evidențiază faptul că nivelul de paralelism în MapReduce este determinat de numărul de splits, nu de numărul maxim de resurse disponibile în cluster. Fiecare mapper procesează datele din split-ul său aplicând aceeași logică de

transformare, dar pe porțiuni diferite de date. Astfel, aceeași operație este executată simultan pe fragmente distincte ale fișierului, ceea ce reprezintă esența procesării distribuite în MapReduce.

Dacă fișierele sunt relativ mici sau împărțite în puține splits, resursele clusterului pot fi subutilizate. Acesta este unul dintre motivele pentru care MapReduce este mai eficient pentru fișiere foarte mari și pentru care, în timp, au apărut soluții mai flexibile de procesare, precum Apache Spark.

Framework-uri comune din ecosistemul Hadoop

Common Hadoop frameworks to process big data			
Apache Flink	Apache Hive	Presto	Apache Pig
• Streaming data flow engine • Uses APIs that are optimized for distributed streaming and batch processing • Provides the ability to perform transformations on data sources • API is categorized into DataSets and DataStreams	• Open-source, SQL-like data warehouse solution • Helps you avoid needing to write complex MapReduce programs in <u>lower level</u> computing languages • Integrates with AWS services such as Amazon S3 and Amazon DynamoDB	• Open-source, in-memory SQL query engine • Emphasizes faster querying for interactive queries • Operates in-memory and is based on its own engine	• Textual data flow language • Performs analysis of large datasets using parallel processing • Supports automatic install when an Amazon EMR cluster is launched • Supports interactive development

 © 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved. 17

Apache Flink este orientat în principal către **procesarea fluxurilor de date în timp real**, fiind potrivit pentru aplicații care necesită reacție rapidă la evenimente, dar poate fi folosit și pentru procesare batch. Datorită modelului său avansat de streaming, Flink este preferat în scenarii cu date continue, cum ar fi senzori sau evenimente de tip log.

Apache Hive este destinat **analizei de tip business** și permite utilizatorilor să interogheze volume mari de date folosind un limbaj asemănător SQL. Hive este foarte util pentru analiști, deoarece elimină necesitatea scrierii de cod MapReduce și oferă o abstracție de nivel înalt pentru lucrul cu datele din Hadoop. **Presto** se concentrează pe **interogări interactive rapide**. Spre deosebire de MapReduce sau Hive clasic, Presto procesează datele în memorie și este ideal pentru explorarea rapidă a datelor și analize ad-hoc, unde timpul de răspuns este critic.

Apache Pig oferă un limbaj de nivel înalt pentru definirea **fluxurilor de procesare a datelor**, fiind mai ușor de utilizat decât MapReduce. Pig este util atunci când se dorește descrierea logicii de transformare a datelor într-o formă mai simplă și mai lizibilă.

<https://www.geeksforgeeks.org/linux-unix/introduction-to-apache-pig/>

Apache Spark

Apache Spark characteristics

- Is an open-source, distributed processing framework
- Uses in-memory caching and optimized query processing
- Supports code reuse across multiple workloads
- Clusters consist of leader and worker nodes



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Apache Spark este un **framework open-source de procesare distribuită**, apărut ca răspuns direct la limitările modelului Hadoop MapReduce, (discutat anterior). Dacă Hadoop a rezolvat problema stocării și a procesării distribuite, Spark a fost conceput pentru a face această procesare **mai rapidă, mai flexibilă și mai ușor de utilizat**. La fel ca Hadoop, Spark rulează într-un mediu distribuit și este capabil să proceseze volume foarte mari de date folosind un cluster de calcul. Diferența fundamentală constă în **modul de execuție** și în **tipurile de workload-uri suportate**.

Procesare distribuită open-source

Spark este un framework open-source, ceea ce înseamnă că poate fi utilizat gratuit și integrat într-o varietate de arhitecturi Big Data. El continuă modelul introdus de **Apache Hadoop**, în care: datele sunt distribuite pe mai multe noduri, calculele sunt executate în paralel, aplicațiile sunt scalabile prin adăugarea de noduri noi. **Spre deosebire de Hadoop MapReduce, Spark nu este limitat la un singur model de procesare batch.**

Procesare in-memory și optimizare

Una dintre cele mai importante caracteristici ale Spark este utilizarea **procesării în memorie (in-memory)**. În timp ce MapReduce scrie rezultatele intermediare pe disc între etapele Map și Reduce, Spark păstrează datele intermediare în memorie ori de câte ori este posibil. Acest lucru: **reduce drastic latența, accelerează execuția joburilor iterative, face Spark mult mai potrivit pentru analize interactive și algoritmi de machine learning.**

Această diferență explică de ce Spark a înlocuit MapReduce în majoritatea scenariilor moderne.

Reutilizarea codului pentru mai multe tipuri de workload-uri

Un alt avantaj major pentru Spark este faptul că **aceiași cod și aceleași structuri de date pot fi reutilizate** pentru: **procesare batch, streaming, interogări SQL, machine learning.**

Acest lucru contrastează cu ecosistemul Hadoop clasic, unde: MapReduce era folosit pentru batch, Hive pentru SQL, Pig pentru fluxuri de date, alte motoare pentru streaming. Spark unifică aceste funcționalități într-o singură platformă coerentă.

Arhitectura clusterului: leader și worker nodes

Din punct de vedere arhitectural, Spark folosește un model **leader-worker**, asemănător conceptual cu ceea ce am văzut la Hadoop și YARN. **Leader node (Driver)**: coordonează execuția aplicației, construiește planul de execuție, distribuie task-urile. **Worker nodes (Executors)**: execută efectiv task-urile, procesează datele, gestionează memoria și calculele locale. Această arhitectură păstrează ideea de coordonare centrală (similară cu Resource Manager / Application Leader din YARN), dar este **mai simplă și mai eficientă**.

Relația cu YARN și Hadoop

Spark poate rula: peste YARN, într-un cluster Hadoop clasic, standalone, peste Kubernetes, sau în platforme complet gestionate.

Astfel, Spark **nu elimină Hadoop**, ci îl poate folosi: HDFS pentru stocare, YARN pentru managementul resurselor (în arhitecturi tradiționale). În multe implementări noi, aceste roluri sunt preluate sau abstractizate.

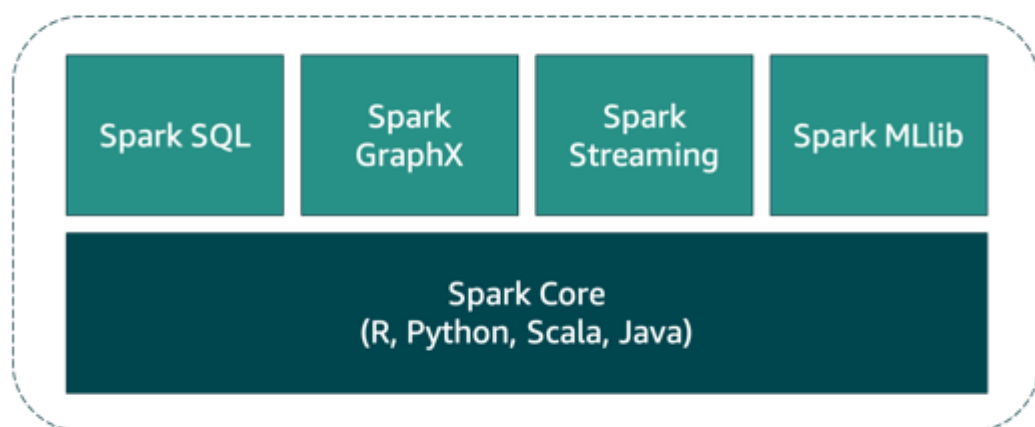
Legătura cu Databricks

Databricks este platforma care a dus Spark la maturitate industrială. Databricks: este construit nativ pe Apache Spark, elimină necesitatea gestionării Hadoop și YARN, oferă clustere automate, scalare dinamică și integrare cu stocare cloud. Astfel, Databricks poate fi văzut ca: **evoluția practică și cloud-native a ideilor introduse de Hadoop și Spark.**

Apache Spark reprezintă **pasul următor în evoluția Big Data**, după Hadoop MapReduce. El păstrează principiile procesării distribuite, dar le îmbunătățește prin procesare in-memory, suport pentru multiple tipuri de workload-uri și o arhitectură mai simplă. Spark stă la baza platformelor moderne precum Databricks și este astăzi motorul dominant pentru procesarea Big Data.

Componentele Apache Spark

Spark components



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Apache Spark este un framework de procesare distribuită conceput ca o **platformă unificată** pentru analiza Big Data. Arhitectura Spark este organizată în jurul unui nucleu

central, numit **Spark Core**, peste care sunt construite mai multe biblioteci specializate.

Această structură modulară permite reutilizarea aceluiași mecanisme de execuție pentru tipuri diferite de workload-uri.

Spark Core – fundamentul platformei

La baza arhitecturii se află **Spark Core**, care reprezintă componenta esențială a framework-ului.

Spark Core este responsabil pentru:

- managementul execuției distribuite,
- planificarea și distribuirea task-urilor,
- gestionarea memoriei,
- toleranța la erori,
- comunicarea între nodurile clusterului.

Spark Core oferă suport pentru mai multe limbaje de programare: **Scala, Python, Java și R**, ceea ce face Spark accesibil pentru categorii diferite de utilizatori. Toate celelalte componente Spark se bazează pe acest nucleu și folosesc aceleași mecanisme de execuție distribuită.

Spark SQL este componenta care permite **procesarea datelor folosind SQL** sau concepte similare bazelor de date relaționale. Aceasta permite interogarea datelor structurate și semi-structurate, oferă optimizări automate ale interogărilor, permite integrarea cu surse de date precum fișiere, tabele sau data lakes. Din punct de vedere didactic, Spark SQL reprezintă evoluția naturală a unor tehnologii precum Hive, dar cu performanțe mult mai bune datorită procesării in-memory.

Spark Streaming permite **procesarea fluxurilor de date în timp real sau aproape de timp real**. Aceasta procesează date care sosesc continuu (loguri, evenimente, senzori), folosește același model de programare ca și procesarea batch, permite unificarea procesării batch și streaming într-o singură aplicație. Astfel, Spark elimină necesitatea utilizării unor motoare separate pentru batch și streaming, așa cum se întâmpla în ecosistemul Hadoop clasic.

Spark MLlib este biblioteca de **machine learning** a platformei Spark care oferă: algoritmi de clasificare, regresie și clustering, suport pentru preprocesarea datelor, posibilitatea de a rula algoritmi ML la scară mare.

MLlib este utilă pentru antrenarea modelelor pe volume mari de date, imposibil de procesat pe un singur sistem.

Spark GraphX este componenta destinată **procesării grafurilor**, unde datele sunt modelate sub formă de noduri și muchii. Aceasta permite analiza relațiilor dintre entități, este utilă pentru aplicații precum rețele sociale, sisteme de recomandare sau analiza dependențelor, combină procesarea grafurilor cu API-urile Spark existente.

Toate aceste componente folosesc **același Spark Core**, rulează pe același cluster, pot fi combinate în aceeași aplicație. Acest lucru reprezintă o diferență majoră față de Hadoop MapReduce, unde fiecare tip de analiză necesita un instrument diferit.

Arhitectura Spark este construită pentru **flexibilitate și performanță**.

Spark Core asigură execuția distribuită, iar componentele superioare – Spark SQL, Streaming, MLlib și GraphX – oferă suport pentru cele mai comune tipuri de analiză Big Data. Această unificare explică de ce Spark a devenit motorul central al platformelor moderne precum Databricks.

**Key takeaways:
Apache Spark**



aws

- Apache Spark performs processing in-memory, reduces the number of steps in a job, and reuses data across multiple parallel operations.
- Spark reuses data by using an in-memory cache to speed up ML algorithms.

© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Acest slide sintetizează caracteristicile esențiale ale **Apache Spark** și explică de ce acest framework a devenit soluția preferată pentru procesarea Big Data în arhitecturile moderne. Mesajul central este legat de **modul în care Spark gestionează datele și execuția joburilor**, comparativ cu soluțiile anterioare, precum Hadoop MapReduce. Un prim aspect fundamental este faptul că Apache Spark realizează **procesarea datelor în memorie (in-memory)**

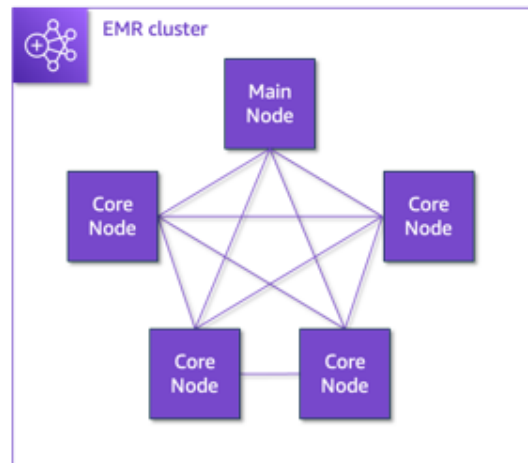
processing). În loc să scrie rezultatele intermediare pe disc, așa cum se întâmplă în MapReduce, Spark păstrează datele în memoria nodurilor de calcul ori de câte ori este posibil. Acest lucru reduce semnificativ numărul de operații de citire și scriere pe disc, care sunt costisitoare din punct de vedere al timpului de execuție, și conduce la performanțe mult mai ridicate. **Un al doilea element important este reducerea numărului de pași necesari pentru executarea unui job.** În Hadoop MapReduce, fiecare etapă Map sau Reduce reprezintă un pas distinct, cu sincronizări și operații intermediare pe disc. Spark permite definirea unor lanțuri de transformări care sunt optimizate și executate eficient ca un întreg. Astfel, logica aplicațiilor devine mai simplă, iar timpul total de execuție scade. De asemenea, Spark oferă posibilitatea de reutilizare a datelor în cadrul mai multor operații paralele. Odată ce un set de date este încărcat și procesat, acesta poate fi păstrat în memorie și folosit în mai multe etape ale aplicației fără a fi recitit din sursa inițială. Această capacitate este deosebit de importantă în scenarii iterative, cum ar fi analizele exploratorii sau algoritmi de machine learning. Slide-ul subliniază explicit rolul **mecanismelor de caching în memorie** în accelerarea algoritmilor de machine learning. În astfel de algoritmi, aceleași date sunt parcurse de mai multe ori pentru ajustarea modelelor. Prin păstrarea datelor în memorie, Spark evită recalculările inutile și reduce drastic timpul de antrenare a modelelor.

Spark - procesare rapidă, eficiență ridicată și reutilizarea inteligentă a datelor.

Schema de clustere și noduri în AWS

Clusters and nodes

- The central component of Amazon EMR is the *cluster*.
- Each instance in the cluster is a *node*.
- The role that each node serves is the *node type*.
- Amazon EMR uses three node types: main, core, and task.



Schema prezentată descrie arhitectura unui cluster **Amazon EMR**, serviciul AWS pentru rularea Hadoop și Spark în cloud. Ideea centrală este că **procesarea Big Data se face într-un cluster**, adică un grup de mașini care colaborează pentru stocarea și procesarea datelor.

Clusterul reprezintă unitatea logică principală. Toate nodurile fac parte din același cluster și comunică între ele. Clusterul este responsabil pentru:

- rularea aplicațiilor Big Data (Spark, Hadoop),
- distribuirea sarcinilor,
- scalarea procesării.

Nodurile

Fiecare mașină din cluster se numește **nod**, iar rolul unui nod este determinat de **tipul de nod**. În Amazon EMR există trei tipuri de noduri, fiecare cu responsabilități clare.

Main node (nodul principal)

Nodul principal este **centrul de coordonare** al clusterului. El gestionează execuția joburilor, rulează serviciile de control (YARN, Spark driver), monitorizează starea clusterului. Acest nod poate fi văzut ca „managerul” care organizează munca celorlalte noduri.

Core nodes - nodurile core reprezintă **infrastructura de bază** a clusterului și stochează date (de exemplu, prin HDFS), execută task-uri de procesare, participă permanent la funcționarea clusterului.

- mai multe noduri core sunt conectate între ele și cu nodul principal, ilustrând procesarea distribuită.

Task nodes - sunt noduri **exclusiv de procesare** care nu stochează date, sunt folosite pentru a adăuga rapid putere de calcul, pot fi adăugate sau eliminate dinamic.

Acest tip de nod permite scalarea eficientă atunci când volumul de lucru crește.

Ce este similar în Databricks

Deși **Databricks** nu folosește explicit noțiunile de *main*, *core* și *task nodes*, **principiile din schema AWS rămân valabile**.

În **Amazon EMR**, rolurile nodurilor sunt: vizibile, configurabile manual, parte din responsabilitatea utilizatorului. În **Databricks**, aceleași roluri există conceptual, dar sunt **gestionate automat** de platformă, utilizatorul nu mai administrează infrastructura, ci se concentrează pe analiză și procesare.

Schema din AWS arată modelul clasic de cluster Big Data, cu roluri explicite pentru noduri.

Databricks păstrează aceeași logică de coordonare și procesare distribuită, dar ascunde complexitatea infrastructurii și automatizează gestionarea clusterului.

Databricks este o **platformă Big Data și AI independentă**, construită pe Apache Spark.

Databrick **nu deține propriile servere**, rulează **deasupra infrastructurii cloud**

Unde este găzduit Databricks

Databricks poate fi rulat pe **trei mari provideri cloud**:

- **Amazon Web Services** → *Databricks on AWS*
- **Microsoft Azure** → *Azure Databricks*
- **Google Cloud** → *Databricks on GCP*

AWS este doar una dintre opțiuni

Amazon EMR

- este **serviciu AWS nativ**
- parte din ecosistemul AWS
- folosește Hadoop, YARN

- utilizatorul vede și configurează infrastructura

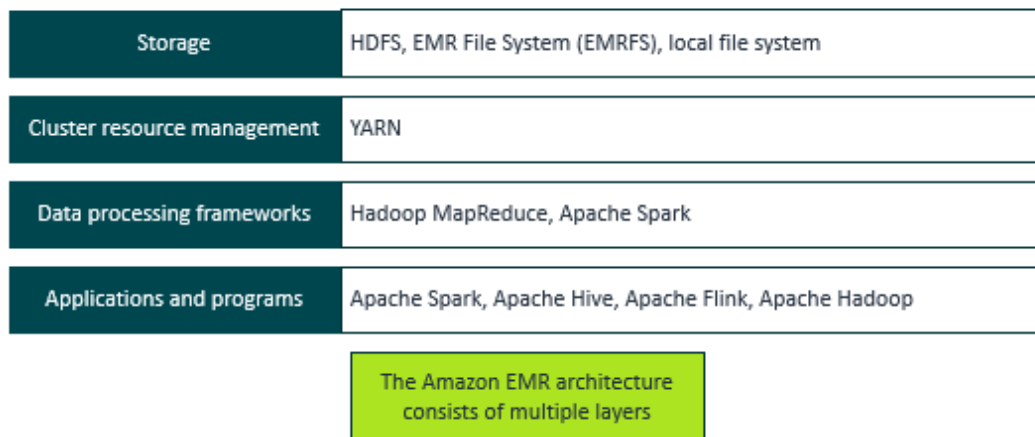
Databricks pe AWS

- este **produs Databricks**
- rulează **pe infrastructura AWS**
- nu folosește Hadoop/YARN
- infrastructura este **ascunsă și automatizată**

Databricks nu este un serviciu AWS, dar poate fi găzduit în AWS.

Este o platformă independentă care folosește infrastructura cloud pentru a rula Apache Spark la scară mare.

Amazon EMR service architecture

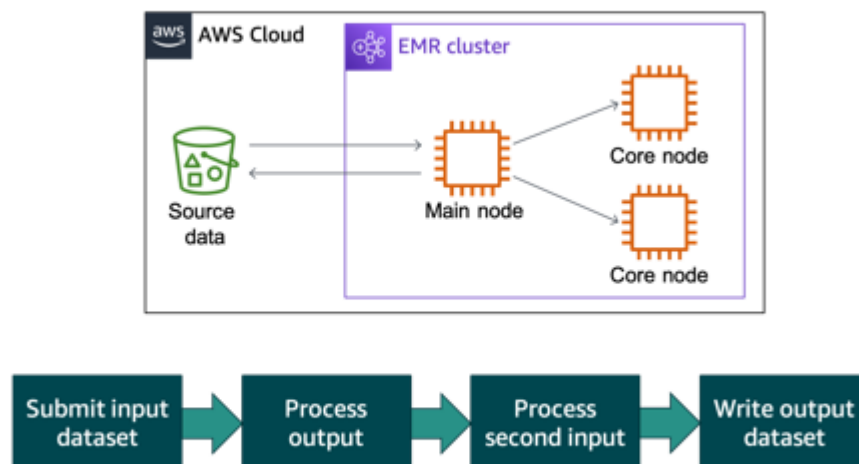


© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Figura prezintă **arhitectura pe straturi a serviciului Amazon EMR**, evidențiind modul în care sunt organizate componentele unui sistem Big Data. La bază se află **stratul de stocare**, care gestionează datele folosind HDFS, EMRFS sau sistemul de fișiere local. Deasupra acestuia se află **stratul de management al resurselor**, reprezentat de YARN, care alocă și coordonează resursele clusterului. Următorul nivel este cel al **framework-urilor de procesare**, precum Hadoop MapReduce și Apache Spark, unde are loc execuția efectivă a calculelor. La nivelul superior se află **aplicațiile și programele** utilizate de utilizatori, cum ar fi Spark, Hive sau Flink, prin care se realizează analiza datelor.

Figura arată că Amazon EMR este construit din mai multe straturi bine separate, fiecare cu un rol clar: stocare, coordonare, procesare și analiză.

Processing data in Amazon EMR



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

fluxul de procesare a datelor în Amazon EMR, de la datele de intrare până la obținerea rezultatelor finale.

Datele sursă sunt stocate în **AWS Cloud** (de exemplu, în Amazon S3) și sunt puse la dispoziția unui **cluster EMR**. În interiorul clusterului, **nodul principal (main node)** coordonează execuția jobului și distribuie sarcinile către **nodurile core**, care realizează efectiv procesarea datelor în mod distribuit. Procesarea are loc în mai multe etape succesive. Mai întâi, setul de date de intrare este trimis spre procesare. Rezultatul acestei prime etape poate deveni, la rândul său, intrare pentru o etapă următoare de procesare. La final, datele procesate sunt scrise ca **set de date de ieșire**, de regulă tot în stocarea din cloud. În ansamblu, figura arată că Amazon EMR funcționează ca o platformă care preia date din cloud, le procesează distribuit într-un cluster și returnează rezultatele, urmând un flux clar: **submit input** → **procesare** → **procesare intermediară** → **scriere output**.

<https://www.databricks.com/discover/pages/optimize-data-workloads-guide>

Shuffle-ul este mecanismul prin care Spark redistribuie datele între noduri pentru a permite procesarea corectă a operațiilor globale, precum agregările și îmbinările. Deși este esențial pentru paralelism, shuffle-ul reprezintă una dintre cele mai costisitoare etape ale procesării

Big Data, motiv pentru care Databricks oferă optimizări dedicate pentru reducerea impactului său asupra performanței.

În procesarea Big Data distribuită, anumite operații necesită **reorganizarea și redistribuirea datelor între nodurile de lucru**, astfel încât datele corelate să poată fi procesate împreună.

Acest mecanism este esențial pentru funcționarea corectă a operațiilor globale, precum agregările și îmbinările de date, și apare atât în modelul Hadoop MapReduce, cât și în Apache Spark. În Apache Spark și în platformele moderne precum Databricks, acest proces poartă denumirea de **shuffle**. Deși termenul este specific Spark, conceptul este implicit prezent și în modelul MapReduce descris anterior.

Redistribuirea datelor în modelul MapReduce

În cadrul modelului MapReduce, după etapa de mapare, datele sunt organizate în funcție de chei. Pentru ca etapa de reducere să poată agrega corect rezultatele, este necesar ca toate valorile asociate aceleiași chei să ajungă pe același nod de lucru.

Acest proces implică:

- transferul datelor între noduri,
- reorganizarea acestora în funcție de cheie,
- pregătirea lor pentru agregarea finală.

Deși cursul descrie aceste etape la nivel conceptual, fără a introduce o terminologie separată, această redistribuire reprezintă exact mecanismul numit *shuffle* în Spark.

Shuffle în Apache Spark și Databricks

În Apache Spark, **shuffle-ul** reprezintă procesul prin care datele sunt mutate între executors pentru a permite efectuarea operațiilor care necesită o vedere globală asupra datelor. Shuffle-ul apare în cazul așa-numitelor **transformări largi**, precum:

- GROUP BY
- JOIN
- DISTINCT
- agregări globale

În contrast, transformările simple, cum ar fi map sau filter, sunt transformări înguste, care nu necesită redistribuirea datelor și pot fi executate local, pe fiecare partiție.

De ce shuffle-ul este o operație costisitoare

Shuffle-ul este considerat una dintre cele mai costisitoare etape ale procesării Big Data, deoarece implică:

- transfer de date prin rețea între noduri,
- scrierea și citirea datelor intermediare pe disc,
- serializarea și deserializarea datelor,
- utilizarea intensivă a memoriei.

Din acest motiv, shuffle-ul reprezintă adesea principalul factor care influențează performanța aplicațiilor Big Data.

Exemplu conceptual: operația DISTINCT

Pentru o operație de tip DISTINCT, procesarea are loc în două etape:

1. **Procesare locală** – fiecare nod determină valorile distincte din partiția sa.
2. **Redistribuire globală** – datele sunt redistribuite între noduri astfel încât fiecare valoare unică să poată fi identificată la nivel global.

A doua etapă implică shuffle și este necesară pentru obținerea unui rezultat corect.

Optimizări moderne în Databricks

Platformele moderne de tip Databricks includ mecanisme avansate pentru reducerea impactului shuffle-ului. Acestea permit:

- minimizarea volumului de date redistribuite,
- optimizarea operațiilor de tip MERGE în Delta Lake,
- controlul gradului de paralelism și al numărului de partiții rezultate.

Prin aceste optimizări, Databricks reușește să păstreze avantajele procesării distribuite, reducând în același timp costurile de execuție.

Identificarea shuffle-ului în practică

În practică, shuffle-ul poate fi identificat prin:

- analiza execuției joburilor în interfața de monitorizare,
- observarea operațiilor de redistribuire a datelor,
- examinarea planului de execuție al interogărilor, unde apar etape dedicate reorganizării datelor.

Redistribuirea datelor între noduri reprezintă un mecanism fundamental al procesării Big Data distribuite. În timp ce modelul MapReduce descrie acest proces la nivel conceptual, Apache Spark și Databricks îl formalizează sub denumirea de shuffle. Deși costisitor din punct de vedere al resurselor, shuffle-ul este indispensabil pentru realizarea operațiilor globale și este optimizat în platformele moderne pentru a asigura performanță și scalabilitate.